

# Tipos de datos abstractos

Notas de clase

Estructuras de datos y algoritmos

Facultad TIC - UPB

Spring 2026

Jorge Mario Londoño Peláez

8 de abril de 2026

## Abstract Data Types

En este capítulo se estudia el concepto de tipo de dato abstracto y su aplicación en la representación de datos de alto nivel que frecuentemente se necesitan en las estructuras de datos y sus algoritmos.



## Índice

|                                                                                                                            |          |
|----------------------------------------------------------------------------------------------------------------------------|----------|
| <b>1. Concepto de Tipos de Datos Abstractos</b>                                                                            | <b>2</b> |
| <b>2. Beneficios de los ADT</b>                                                                                            | <b>2</b> |
| <b>3. Implementación de ADT en la Programación Orientada a Objetos (POO)</b>                                               | <b>3</b> |
| <b>4. Ejemplos básicos de ADT</b>                                                                                          | <b>3</b> |
| <b>5. Prueba unitaria de las operaciones de un ADT</b>                                                                     | <b>4</b> |
| <b>6. Uso de la herencia</b>                                                                                               | <b>4</b> |
| 6.1. Tipos de herencia . . . . .                                                                                           | 5        |
| 6.2. La interface como la definición del Application Programming Interface (API) del<br>Abstract Data Type (ADT) . . . . . | 5        |
| 6.3. La herencia de implementación . . . . .                                                                               | 5        |
| 6.4. Ejemplos de herencia en Java . . . . .                                                                                | 6        |
| <b>7. Métodos especiales en Python</b>                                                                                     | <b>8</b> |
| 7.1. Métodos Especiales Comunes . . . . .                                                                                  | 8        |
| 7.2. Ejemplo en Python . . . . .                                                                                           | 8        |

## 1. Concepto de Tipos de Datos Abstractos

En esta sección se introducen los conceptos fundamentales de los Tipo de Dato Abstracto (TDA) (ADT). Para comprender qué es un ADT, es útil considerar una jerarquía de niveles de representación de datos, desde el más bajo (bits) hasta el más abstracto:

- **Representaciones de bajo nivel:** En el nivel más básico, los datos se representan como secuencias de bits (datos binarios). Estas representaciones son directamente interpretadas por el hardware de la computadora.
- **Tipos de datos primitivos (o concretos):** Son los tipos de datos básicos que los lenguajes de programación ofrecen directamente, y que tienen un soporte nativo en el hardware. Ejemplos comunes incluyen `int` (enteros), `float` (números de punto flotante), `char` (caracteres) y `bool` (booleanos).
- **Tipos de datos abstractos:** A partir de los tipos primitivos, se pueden construir estructuras más complejas. Los ADT son modelos de datos que se definen en términos de las operaciones que se pueden realizar sobre ellos, en lugar de cómo se almacenan en la memoria. Un ADT encapsula la representación de los datos y las operaciones permitidas, ofreciendo una interfaz clara y bien definida. Los ADT se construyen utilizando tipos de datos primitivos u otros ADT más simples, permitiendo modelar los datos de una manera más cercana a la lógica del problema que se está resolviendo.

## 2. Beneficios de los ADT

Los ADT ofrecen varios beneficios importantes en el desarrollo de software, que mejoran la calidad, la mantenibilidad y la reutilización del código:

- **Alto nivel de abstracción:** Los ADT permiten modelar los datos de una manera que se asemeja mucho a la forma en que se conciben en el problema real. Esto facilita la comprensión del problema y el diseño de la solución, ya que se trabaja con conceptos más cercanos al dominio del problema.
- **Modularidad y Reutilización:** Los ADT fomentan la creación de componentes de software reutilizables. Una vez que se define un ADT, se puede utilizar en diferentes partes de un programa o incluso en diferentes proyectos, lo que reduce la duplicación de código y acelera el desarrollo.
- **Encapsulación y Ocultamiento de la Información:** Los ADT encapsulan la representación interna de los datos y exponen una interfaz bien definida para interactuar con ellos. Esto oculta los detalles de implementación y protege los datos de manipulaciones incorrectas, lo que ayuda a mantener la consistencia y la integridad de los datos.
- **Facilidad de Mantenimiento:** Al separar la interfaz de la implementación, los ADT facilitan el mantenimiento y la evolución del software. Los cambios en la implementación de un ADT no afectan al resto del código, siempre y cuando la interfaz se mantenga igual.

### 3. Implementación de ADT en la Programación Orientada a Objetos (POO)

En la Programación Orientada a Objetos (POO) (Object Oriented Programming (OOP)), los ADT se implementan utilizando clases y objetos. Esta sección detalla cómo se lleva a cabo esta implementación:

- **Representación del Estado del TDA:** El estado de un ADT se representa mediante las variables de instancia (también conocidas como atributos o campos) dentro de una clase. Estas variables almacenan los datos que definen el estado específico de cada objeto del ADT.
- **Interfaz del ADT (API):** La interfaz de un ADT se define a través de los métodos públicos de la clase. Estos métodos son las operaciones que se pueden realizar sobre los objetos del ADT, y permiten interactuar con los datos de manera controlada y segura.
- **Clases como Implementación de TDA:** En OOP, un ADT se implementa como una clase. La estructura del ADT se define por los tipos de datos de las variables de instancia, mientras que el comportamiento se define por los métodos de la clase.
- **Objetos como Instancias del ADT:** Los datos del tipo del ADT se representan como instancias de la clase, es decir, objetos. Cada objeto es una realización concreta del ADT, con su propio estado específico.

### 4. Ejemplos básicos de ADT

A continuación, se presentan algunos ejemplos básicos de ADT implementados en Python, mostrando cómo se definen las clases y sus métodos para encapsular los datos y las operaciones.

Antes de ver los ejemplos, es importante tener en cuenta una particularidad de Python: no existen modificadores de acceso como `private` o `protected`. Por convención, los atributos que se consideran internos se nombran con un prefijo de guión bajo (por ejemplo, `_valor`), aunque el lenguaje no impide el acceso directo a ellos. En los siguientes ejemplos se usan atributos públicos por simplicidad.

- **ADT Fecha:** Representa una fecha con día, mes y año.

```
1 class Fecha:
2     def __init__(self, dia, mes, anno):
3         self.dia = dia
4         self.mes = mes
5         self.anno = anno
6
7     def __str__(self):
8         return f"{self.dia}/{self.mes}/{self.anno}"
```

- **ADT Contador:** Representa un contador con un nombre y un valor entero.

```
1 class Contador:
2     def __init__(self, nombre, valor=0):
3         self.nombre = nombre
4         self.valor = valor
5
6     def incrementar(self):
7         self.valor += 1
```

```

8
9     def __str__(self):
10         return f"{self.nombre}: {self.valor}"

```

- **ADT Punto:** Representa un punto en un plano cartesiano con coordenadas x e y.

```

1 class Punto:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def mover(self, dx, dy):
7         self.x += dx
8         self.y += dy
9
10    def __str__(self):
11        return f"({self.x}, {self.y})"

```

## 5. Prueba unitaria de las operaciones de un ADT

Las pruebas unitarias son una parte fundamental del desarrollo de software, ya que permiten verificar que cada componente del programa (en este caso, las operaciones de un ADT) funciona correctamente de forma aislada. En esta sección, explicaremos cómo implementar pruebas unitarias sencillas utilizando sentencias **assert** en Python, sin necesidad de librerías especializadas.

Las pruebas unitarias se basan en la idea de crear casos de prueba que ejerciten diferentes caminos de ejecución del código y verificar que el comportamiento observado coincide con el comportamiento esperado. Cada caso de prueba debe probar una funcionalidad específica del ADT.

A continuación, se muestra un ejemplo de cómo implementar una prueba unitaria para el método **mover** del ADT **Punto**:

```

1 # Prueba unitaria del metodo mover
2 p = Punto(1, 2)
3 assert str(p) == "(1, 2)" # verificar el estado inicial
4 p.mover(3, 4)
5 assert str(p) == "(4, 6)" # verificar el estado despues de mover
6 print("Prueba unitaria del metodo mover: OK")

```

En este ejemplo, primero se crea un objeto **Punto** con coordenadas iniciales (1, 2). Luego, se utiliza una sentencia **assert** para verificar que la representación en cadena del punto es la esperada. A continuación, se llama al método **mover** para desplazar el punto y se utiliza otra sentencia **assert** para verificar que las coordenadas del punto se han actualizado correctamente. Si alguna de las aserciones falla, el programa se detendrá con un error. Si todas las aserciones pasan, se imprimirá un mensaje indicando que la prueba unitaria ha sido exitosa.

Este enfoque sencillo permite verificar el comportamiento de las operaciones de un ADT de manera efectiva y sin necesidad de herramientas complejas. El ejemplo anterior muestra un caso básico; en la práctica, es importante crear casos de prueba adicionales para cubrir diferentes escenarios, por ejemplo: mover con desplazamiento cero, mover con valores negativos, o verificar que dos movimientos consecutivos acumulan correctamente las coordenadas.

## 6. Uso de la herencia

La herencia es un concepto fundamental en la POO que permite crear nuevas clases basadas en clases existentes. Esto promueve la reutilización de código y la creación de jerarquías de clases.

En el contexto de los ADT, la herencia se puede utilizar de dos maneras principales: herencia de interfaces y herencia de implementación.

Para ilustrar estos conceptos, usaremos Java en los siguientes ejemplos, ya que este lenguaje distingue explícitamente entre interfaces (`interface`) y clases (`class`), lo que hace más visible la diferencia entre ambos tipos de herencia.

### 6.1. Tipos de herencia

En la POO, la herencia se manifiesta principalmente de dos formas:

- **Herencia de interfaces:** Se enfoca en heredar las firmas de los métodos (es decir, los nombres, parámetros y tipos de retorno) definidos en una clase abstracta o interfaz, sin heredar la implementación.
- **Herencia de implementación:** Se hereda tanto la estructura como la implementación de una clase base.

### 6.2. La interface como la definición del API del ADT

Una interfaz define el API de un ADT, especificando las operaciones que se pueden realizar sobre los datos, sin revelar cómo se implementan estas operaciones. En esencia, una interfaz es un contrato que las clases deben cumplir.

El tipo de una interfaz se puede utilizar como un tipo de datos, lo que permite el polimorfismo. Esto significa que diferentes clases pueden implementar la misma interfaz, y el código cliente puede interactuar con estas clases a través de la interfaz, sin conocer la implementación específica. Esto facilita la creación de sistemas flexibles y extensibles. El Ejemplo 1 a continuación ilustra este principio: el código cliente puede declarar una variable de tipo `Punto2D` y asignarle tanto un `Punto2DCartesiano` como un `Punto2DPolar`.

### 6.3. La herencia de implementación

En algunos casos, es útil heredar la implementación de una clase base, además de su interfaz. Esto permite reutilizar el código de la clase base y evitar la duplicación de código.

Un ejemplo concreto de herencia de implementación se encuentra en Java, donde todas las clases heredan implícitamente de la clase `Object`. Esto significa que cualquier clase que se cree ya posee una implementación por defecto de ciertos métodos fundamentales:

- **equals:** Permite comparar dos objetos para determinar si son iguales. Es importante sobrescribir este método para que la comparación se base en el contenido del objeto (el valor del ADT), en lugar de comparar sus referencias.
- **toString:** Devuelve una representación en cadena del objeto. Es útil para la depuración y la visualización de los datos. Idealmente todo ADT debe proporcionar la forma de representar textualmente el valor del ADT para por ejemplo visualizarlo fácilmente.
- **hashCode:** Devuelve un código hash para el objeto. Este método es importante para el uso de ADT en estructuras de datos como las tablas asociativas.

## 6.4. Ejemplos de herencia en Java

**Ejemplo 1: ADT Punto2D – Herencia de interfaz** El siguiente ejemplo muestra cómo definir un ADT mediante una interfaz Java y proporcionar dos implementaciones distintas. Observe que ambas clases implementan los mismos métodos pero con representaciones internas diferentes (coordenadas cartesianas y polares), lo que ilustra el polimorfismo descrito anteriormente.

```
1 interface Punto2D {
2     double getX();
3     double getY();
4     double distancia(Punto2D otro);
5 }
```

Listing 1: Interfaz Punto2D

```
1 class Punto2DCartesiano implements Punto2D {
2     private double x;
3     private double y;
4
5     public Punto2DCartesiano(double x, double y) {
6         this.x = x;
7         this.y = y;
8     }
9
10    @Override
11    public double getX() {
12        return x;
13    }
14
15    @Override
16    public double getY() {
17        return y;
18    }
19
20    @Override
21    public double distancia(Punto2D otro) {
22        double dx = this.getX() - otro.getX();
23        double dy = this.getY() - otro.getY();
24        return Math.sqrt(dx * dx + dy * dy);
25    }
26 }
```

Listing 2: Implementación cartesiana de Punto2D

```
1 class Punto2DPolar implements Punto2D {
2     private double radio;
3     private double angulo;
4
5     // angulo en radianes (estandar en Java)
6     public Punto2DPolar(double radio, double angulo) {
7         this.radio = radio;
8         this.angulo = angulo;
9     }
10
11
12    @Override
13    public double getX() {
14        return radio * Math.cos(angulo);
15    }
16 }
```

```
16
17     @Override
18     public double getY() {
19         return radio * Math.sin(angulo);
20     }
21
22     @Override
23     public double distancia(Punto2D otro) {
24         double dx = this.getX() - otro.getX();
25         double dy = this.getY() - otro.getY();
26         return Math.sqrt(dx * dx + dy * dy);
27     }
28 }
```

Listing 3: Implementación polar de Punto2D

**Ejemplo 2: Implementación de equals** Este ejemplo muestra cómo implementar el método `equals` para la clase `Punto2D`. La implementación debe comparar los puntos con una tolerancia para tener en cuenta la precisión limitada de los números de punto flotante.

```
1     @Override
2     public boolean equals(Object obj) {
3         if (this == obj) return true;
4         if (obj == null || !(obj instanceof Punto2D)) return false;
5         Punto2D other = (Punto2D) obj;
6         return distancia(other) < TOL;
7     }
8
9     static final double TOL = 1E-12;
```

Listing 4: Implementación de equals en Punto2D

**Nota:** La igualdad basada en distancia con tolerancia no es estrictamente transitiva: si `a.equals(b)` y `b.equals(c)`, no necesariamente `a.equals(c)`. Sin embargo, es una aproximación práctica para manejar la imprecisión inherente de la aritmética de punto flotante. El valor de `TOL` debe seleccionarse apropiadamente según los márgenes de error de las operaciones de punto flotante involucradas.

**Ejemplo 3: Implementación de toString** Este ejemplo muestra cómo implementar el método `toString` para las clases `Punto2DCartesiano` y `Punto2DPolar`.

```
1     @Override
2     public String toString() {
3         return "(" + x + ", " + y + ")";
4     }
```

Listing 5: Implementación de toString en Punto2DCartesiano

```
1     @Override
2     public String toString() {
3         return "(" + radio + "@" + angulo + ")";
4     }
```

Listing 6: Implementación de toString en Punto2DPolar



**Ejemplo 4: Prueba unitaria** Este ejemplo muestra cómo realizar una prueba unitaria para verificar el correcto funcionamiento del método `equals`.

```
1 public static void main(String[] args) {  
2     Punto2D p1 = new Punto2DCartesiano(1, 1);  
3     Punto2D p2 = new Punto2DCartesiano(1.00001, 1.00001);  
4     Punto2D p3 = new Punto2DCartesiano(2, 2);  
5     Punto2D p4 = new Punto2DPolar(Math.sqrt(2), Math.PI / 4);  
6  
7  
8     assert p1.equals(p2) : "p1 debe ser igual a p2";  
9     assert !p1.equals(p3) : "p1 no debe ser igual a p3";  
10    assert p1.equals(p4) : "p1 debe ser igual a p4";  
11  
12    System.out.println("Pruebas unitarias pasaron");  
13 }
```

Listing 7: Prueba unitaria de equals

## 7. Métodos especiales en Python

En la sección anterior vimos cómo Java utiliza la herencia de la clase `Object` para proporcionar métodos como `equals` y `toString`. Python ofrece un mecanismo análogo a través de los métodos especiales.

Los métodos especiales, también conocidos como **métodos mágicos** o **dunder methods** (por sus nombres que comienzan y terminan con doble guión bajo), son métodos que permiten personalizar el comportamiento de los objetos en Python. Estos métodos son invocados automáticamente por el intérprete en ciertas situaciones, como cuando se crea un objeto, se imprime, se compara, etc.

En la implementación de TDAs, los métodos especiales son muy útiles para definir cómo se comportan los objetos de nuestro TDA en diferentes contextos.

### 7.1. Métodos Especiales Comunes

- `__init__`: El constructor de la clase. Se llama cuando se crea un nuevo objeto de la clase.
- `__str__`: Define cómo se representa un objeto como una cadena de texto. Se llama cuando se usa la función `str()` o `print()` sobre el objeto.
- `__eq__`: Define cómo se comparan dos objetos para determinar si son iguales. Se llama cuando se usa el operador `==`.
- `__hash__`: Define el valor hash de un objeto. Es necesario para usar objetos como claves en diccionarios o en conjuntos. Cuando se sobre-escribe el método `hash`, es importante mantener la consistencia con el `__eq__`: Si dos objetos son iguales, sus hashes deben ser los mismos.

Estos métodos corresponden directamente a los métodos de la clase `Object` en Java: `__eq__` equivale a `equals`, `__str__` equivale a `toString`, y `__hash__` equivale a `hashCode`.

### 7.2. Ejemplo en Python

A continuación, se muestra un ejemplo de cómo usar estos métodos especiales en el ADT `Punto`:

```
1 class Punto:
2     def __init__(self, x, y):
3         self.x = x
4         self.y = y
5
6     def __str__(self):
7         return f"({self.x}, {self.y})"
8
9     def __eq__(self, otro_punto):
10        if otro_punto is None:
11            return False
12        if isinstance(otro_punto, Punto):
13            return self.x == otro_punto.x and self.y == otro_punto.y
14        return False
15
16    def __hash__(self):
17        return hash((self.x, self.y))
```

En este ejemplo, hemos definido cómo se crea un **Punto**, cómo se representa como una cadena, cómo se compara con otro **Punto** y cómo se calcula su valor hash. Esto nos permite usar los objetos **Punto** de manera más natural y consistente.

**Nota:** En este ejemplo se utiliza igualdad exacta para simplificar. En aplicaciones que involucren aritmética de punto flotante, sería necesario implementar una comparación con tolerancia, como se mostró en el ejemplo de Java.

## Resumen

Los TDA permiten modelar datos de alto nivel independientemente de su representación interna. Tanto Java (mediante interfaces y la herencia de **Object**) como Python (mediante métodos especiales) proporcionan mecanismos para definir la interfaz y el comportamiento de un TDA. La práctica de acompañar cada TDA con pruebas unitarias asegura que las operaciones se comporten según lo esperado.